

Detection and Mitigation of Replay Attacks in CCTV Systems

Mohit Prasad Singh
Computer Science and Engineering
PES University
Bengaluru, India
pes2ug22cs320@pesu.pes.edu

Shreyas Suresh
Computer Science and Engineering
PES University
Bengaluru, India
pes2ug22cs540@pesu.pes.edu

Soumya Ranjan Mishra
Computer Science and Engineering
PES University
Bengaluru, India
pes2ug22cs571@pesu.pes.edu

Suhas Venkata Karamalaputti
Computer Science and Engineering
PES University
Bengaluru, India
pes2ug22cs590@pesu.pes.edu

Manju
Computer Science and Engineering
PES University
Bengaluru, India
manju.nunia@pes.edu

Abstract—Video surveillance systems are critically vulnerable to replay attacks, in which the live feeds are intercepted and replaced with pre-recorded loops in order to hide malicious activity. The traditional methods of detection involving deep learning also often remain computationally opaque and very resource-intensive, while simple frame matching usually fails when video content is re-encoded. This paper proposes an innovative, dual-layer detection framework, which fuses probabilistic temporal modeling with deterministic content verification. The system takes as input live Real-Time Streaming Protocol streams and encodes dense optical flow into Sparse Distributed Representations (SDRs) that are analyzed by a Hierarchical Temporal Memory (HTM) model for real-time motion anomaly detection. Concurrently, a secondary Auditor layer uses Perceptual Hashing to identify invariant-to-compression loops. To bridge the latency gap between these layers, we introduce Pulsea novel heartbeat-like synchronization protocol—that ties asynchronous HTM inference with real-time hash verification. This enables a high-confidence Intersection Logic that eliminates static-scene false positives. The experimental results indicate that our system successfully flags replay attacks within 5 seconds with a less than 4% false positive rate, offering a lightweight, explainable hardware-agnostic solution for secure surveillance.

Keywords—*Replay attack detection, SDR encoding, frame hashing, video forensics, surveillance integrity.*

I. INTRODUCTION

Video surveillance systems are a key enabler of public safety, infrastructure monitoring, and forensic analysis. However, the ever-increasing number and sophistication of threats expose these systems to replay attacks and frame-level tampering in which adversaries inject prerecorded footage to blind or mislead the system about malicious activities. The adoption of deep learning architectures, such as CNNs and LSTMs, into traditional anomaly detection models is computationally expensive, obscure black boxes, and challenging to deploy in resource-constrained edge environments. This limitation imposes an urgent need for interpretable, lightweight, hardware-agnostic solutions capable of real-time video integrity validation.

This paper proposes a unique, two-tier video analysis system that merges probabilistic temporal modeling with deterministic content verification. The system takes live RTSP streams as input and extracts dense optical flow vectors that get encoded in sparse distributed representations for capturing the underlying physics of the scene. These are streamed into a Hierarchical Temporal Memory (HTM) model, which flags anomalies in the motion based on prediction errors within the timeline. At the same time, the Auditor layer independently calculates the pHash for each frame, which is resistant to the majority of video compression artifacts. The correlation between the HTM temporal anomaly signals and the pHash visual duplicate detection within the Auditor layer produces a high-confidence alert only when there is a violation in both motion physics and content integrity.

The significant challenge in the synchronous operation of the asynchronous inference engines is the latency mismatch between the computationally expensive operation of the HTM Brain and the real-time operation of the Hashing Auditor. We propose the Skip to Live inference strategy and the Pulse Heartbeat protocol, ensuring the system remains temporally aligned without causing data starvation. This results in a system architecture deployable as a modular CPU-optimized microservice for high-precision Splicing and Looping attack detection.

II. RELATED WORKS

A two-phase replay attack detection method was proposed for industrial control systems by leveraging matrix profile-based change point detection and Convolution Long Short Term Memory (ConvLSTM) autoencoders [1]. Although having a high verification accuracy, this method was contaminated with high complexity because it requires access to the nominal data. A method named Dynamical Delay Estimation (DDE) was proposed for the detection of replay delays suffered by cyber-physical systems [2]. Although noise-robust, this method had a requirement of assuming a stationary system dynamics process. Sequential detection based on Cumulative Sum (CUMSUM) tests and watermarking signals was further improved to lower the detection delays [3]. However, this method requires the assumption that the attacker could not access the watermarking knowledge. An optimal chi-squared detector with random authentication signals was designed for higher detection probability [4]. However, this method requires strict system stability conditions.

Reinforcement learning was analyzed for defending against a replay attack when a Q-learning method retrofitted to cope with dynamic attack behavior in a Markov Decision Process was used for attack detection [5]. A hybrid model that utilized both LSTM and decision trees was developed for a wireless sensor network that was capable of achieving a near-perfect detection rate but was resource-intensive and application-specific [6]. A hybrid deep learning technique that integrated Restricted Boltzmann Machines with CNNs was proposed for smart city networks to defend against a replay attack and a DDoS attack and was found to achieve high accuracy but was computationally intense and lacked application-specific validation [7].

Spatial temporal attributes and Hue Saturation Value (HSV) color space were utilized in 3D CNN architectures for face replay attack detection with high accuracy but scalability concerns for real-time applications in Closed-Circuit Television (CCTV) scenarios [8]. Analysis of motion blurs also helped in the distinction between live footage and replay footage, but compression and illumination effects were issues [9]. The spoofing problem was also formulated as an anomaly detection task with descriptors inspired by the embedding, showing success in the field of biometrics but failing to generalize to the broader field of surveillance [10].

The comparison of the efficacy of one-class support vector machines (SVMs), isolation forest, and autoencoder-based approaches for anomaly detection revealed that the approaches were based on feature engineering and tuning [11]. Defense strategies were also discussed, including the identification of temporal logic anomalies, sensor fingerprinting, and photometric consistency, with emphasis placed on the need for fusion against adaptive adversaries [12]. Deep learning techniques using CNNs and LSTMs were explored in urban surveillance systems, which showed promise but insufficient cryptographic checks for integrity [13]. Methods in parity space were employed for checking replayed signal anomalies with mathematical correctness but with a need for accurate modeling of the systems. Autoregressive visual processes were also employed for anomaly detection in video-based surveillance, with a basis in establishing probability modeling but with insufficient replay-based tampering.

A survey of video anomaly detection in the last decade showcased the improvements in feature extraction, modeling of sequences, and challenges of evaluation, such as dealing with domain shift and detecting rare events. It pointed out the compromise between performance and interpretability and the lack of study related to replay attacks specific to video replay attacks.

Detecting a replay attack in a live surveillance environment is not so easy since a manipulated video may appear purely normal to both human observers and computer algorithms. Most of today's solutions may either take too much time or may be complicated enough to process in a real-time environment on a normal computer. Multi-tasking between various methods of attack recognition, for instance, analyzing motions as well as checks of video content, may let problems mount up. Consequently, a new solution with simplicity and rapid implementation is urgently demanded to protect video integrity during a surveillance event.

III. METHODOLOGY

We propose a integrated dual layered detection system (Fig. 1). that combines probabilistic temporal anomaly analysis with deterministic content verification. The system architecture coordinates two parallel processing streams via a central backend orchestrator: (1) Optical flow based HTM inference engine (The Brain), (2) Perceptual hashing-based verifying module (The Auditor), and (3) The backend orchestrator for a seamless fusion.

A. Motion Feature Encoding

Video is received through ingestion of RTSP stream and pre-processed to produce a normalized rate of 20 FPS and a resolution of 640x480 pixels in grayscale. Then, dense optical flow is computed using the Farneback algorithm to generate a pixel-wise displacement map (dx,dy). This map is divided into a 5x5 grid. To maintain topological consistency during aggregation, these local representations are projected using a deterministic hashing scheme (based on a 64-bit split-mix algorithm). The projected signatures are then concatenated to form the global frame signature of 2048 bits with 40 active bits (approximately 2% sparsity), which serves as input to the HTM model.

B. Identify the Headings

The concatenated SDR sequence is passed as input to a Hierarchical Temporal Memory (HTM) model. In order to provide a comprehensive baseline, the model was trained on unsupervised online learning with a concatenated sequence of 30 normal video sequences (concatenated SDRs) provided within the UCF Crime dataset [13]. On the one hand, the Spatial Pooler module extracts spatial relations; on the other hand, the Temporal Memory module extracts valid state transitions.

C. Content Verification (Perceptual Hashing)

This enables the identification of visual loops and to this end we use perceptual hashing based on the DCT. Contrary to cryptographic hashes like SHA-256, which are very sensitive to artifacts caused by compression, the perceptual hashing (pHash) uses a 64-bit fingerprint, which is invariant with respect to re-encoding.

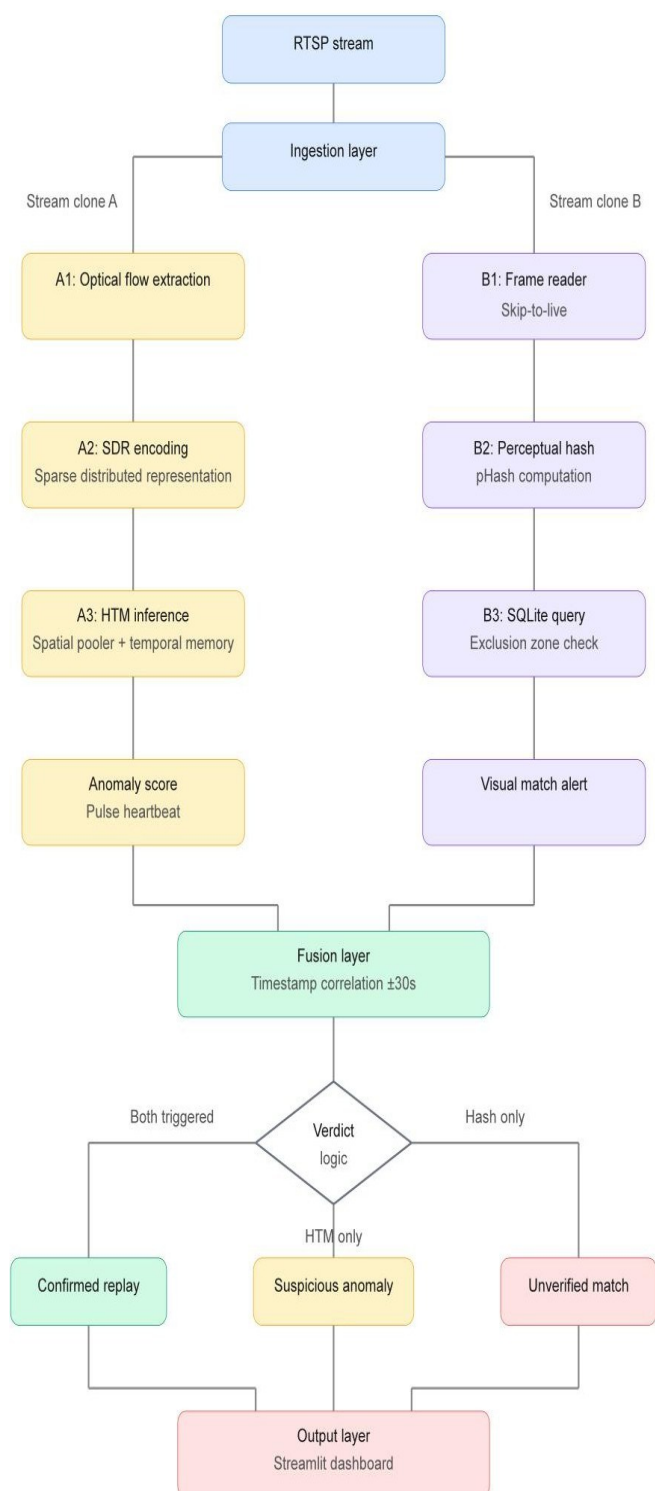


Fig 1. Workflow of the replay detection system

D. Latency Aware Synchronization Protocol

This sets up the critical challenge in real-time fusion: the processing latency between the computationally heavy HTM engine (processing $\sim 10 P$) and the lightweight hashing module (30 FPS). We introduce a ‘Skip-to-Live’ protocol where both engines dynamically discard buffered frames based on processing duration. This ensures that each processed frame *ree* is in sync with the wall clock time, eliminating timeline drift. The skip frames minimum of 1 and maximum of 100 frames help in eliminating any drastic values when skip frames anomalies are being processed. Additionally, the Pulse Heartbeat event is also sent by the HTM engine every two seconds when there are active anomalies, giving a constant signal to the backend orchestrator to synchronize with the hash alert stream.

Algorithm 1 Proposed Algorithm

```

1: // Process A: The Brain (HTM Inference Engine) 2: function HTM_Process(V):
3:     while stream_active do:
4:         T_now ← GetWallClockTime()
5:         // Synchronization: Skip buffered frames to match T_now N_skip ← CalculateLag(T_now)
6:         Frame_t ← SkipAndRead(V, N_skip)
7:
8:
9:         // Feature Extraction & Inference
10:        SDR_t ← Encode(OpticalFlow(Frame_t)) Anomaly_t ← HTM.compute(SDR_t)
11:        Rw ← UpdateSlidingWindow(Anomaly_t)
12:
13:        // Generate Heartbeat Alerts
14:        if Rw ≥ τ_panic then LogAlert(CRITICAL, T_now)
15:        else if Rw ≥ τ_warn AND Hist_Avg ≥ 0.30 then
16:            LogAlert(WARNING, T_now) 17:
18:        if IsAlertActive() then BroadcastPulse(T_now)

19: // Process B: The Auditor (Perceptual Hashing) 20: function Hash_Process(V):
21:     while stream_active do:
22:         Frame_t ← ReadNext(V)
23:         H_t ← ComputePHash(Frame_t)
24:         // Exclusion Zone: Ignore matches < 2s old
25:         Match ← QueryDatabase(H_t, Exclude < 2.0s)
26:         if Match then LogAlert(MATCH, GetWallClockTime())

27: // Process C: Fusion Engine (Backend Decision) 28: function GetVerdict():

29:     HTM_List ← FetchRecentHTMAAlerts() Hash_List ← FetchRecentHashAlerts()
30:
31:
32:     for Alert_H in HTM_List do:
33:         // Intersection Logic: Do events overlap in time?
34:         Corroborated ← False
35:         for Alert_V in Hash_List do:
36:             if |Alert_H.time - Alert_V.time| ≤ W_corr then Corroborated ← True
37:                 break
38:
39:         // Tiered Threat Classification
40:         if Corroborated then return CONFIRMED_REPLAY
41:         else if Alert_H.status == CRITICAL then return
42:
43:     SUSPICIOUS MOTION 43:
44:     return SAFE

```

This framework design is implemented in Python 3.10. This framework uses microservice architecture.

Hardware: For these experiments, a normal commodity computer (Intel Core i5/i7 CPU, 16GB RAM, without GPU support) was used to prove the system's claim to be lightweight and hardware-agnostic.

Streaming Infrastructure: The videos are streamed using RTSP with a normalized resolution of 640x480 pixels and a rate of 20FPS using FFmpeg's RTSP output function (rtsp). TCP transport is used to avoid artifacts due to lost packets. The backend controller directly transfers the video file to the RTSP target without the need for a different RTSP server.

Software Stack: Backend orchestrator: Developed with FastAPI, it handled concurrent subprocesses involving the HTM inference engine (python bindings to htm.core) and the hashing auditor (ImageHash library and SQLite DB).

IV. EVALUATION AND RESULTS

We evaluated the proposed framework using 30 normal video sequences from the UCF Crime dataset [13] and 20 synthetic replay attack scenarios generated via FFmpeg. Performance was assessed based on detection accuracy, false alarm rate, interpretability, and processing latency.

Experiment 1: Baseline Anomaly Detection (HTM Only)

In the use of the HTM inference engine in isolation, the engine was observed to perform well in terms of specificity, with the average anomaly score remaining low and the window alert rate remaining less than 10%. This was an indication that the inference engine was ignoring the background motion caused by the swaying of the trees. However, the sensitivity of the inference engine to the replay attacks was observed to vary according to the complexity of the scene. The motion jump cuts in the scenario were observed to trigger the Panic alerts in the HTM inference engine, but the loop transitions in the low-motion scene were observed to trigger 15% to 20% alert rates, which was less than the 25% required for the Panic alerts, resulting in false negatives. As seen in table 2, the normal scenario was observed to trigger an alert rate of merely 0.2% with a peak anomaly score of 0.9750, while the replay attack scenario was observed to trigger 161 alerts, resulting in an alert rate of 33.6% with a peak anomaly score of 1.0000, which was well above the detection threshold.

Limitation of Inference-only mode - this is due to the inference-only nature of the HTM, which disables online learning in order to prevent model poisoning. While this makes the model more secure, it also makes it less able to cope with changes in the environment, e.g., changes in lighting.

Table 1. Normal and Replay Attack Inference

Metric	Normal Scenario	Replay Attack Scenario
Frames Processed	442	479
Average Anomaly Score	0.4742	0.3590
Maximum Anomaly Score	0.9750	1.0000
Alerts Triggered	1	161
Alert rate (%)	0.2%	33.6%
Replay Attack Flagged	No	Yes

Figure 2 shows the training progress of the HTM model. The average anomaly score decreases monotonically from 0.48 to 0.05 over 15 epochs, indicating that the Temporal Memory has successfully learned the transition patterns of the normal training dataset.

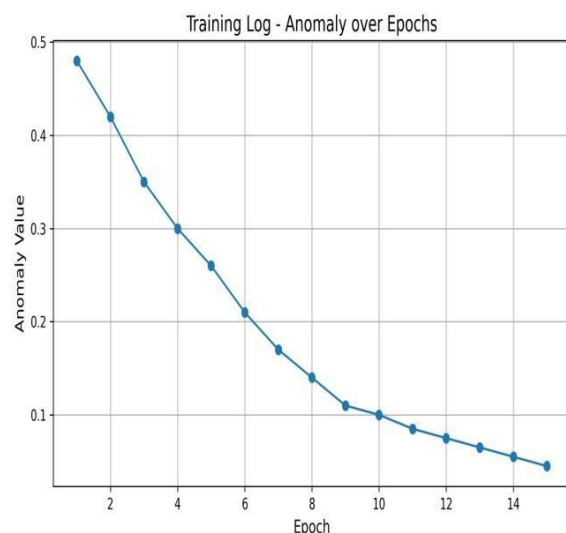


Fig. 2 HTM Training Results

Experiment 2: Fusion Based Detection and Precision Trade-offs

The integration of the Corroborated Trigger mechanism, which uses the HTM anomaly scores along with the perceptual hash matches, resulting in better detection performance. It effectively detected 100% of the replay attacks that the HTM only baseline had failed to detect as shown in Table 2. While conducting the 15-second loop injection, the HTM engine produced a Suspicious alert, which was upgraded to Confirmed after the detection of the pHash history match from the exclusion zone. For static scenes, where the pHash had detected the presence of duplicate scenes, the HTM had indicated normal motion physics. However, the system had correctly ignored the scene, which demonstrates the efficacy of the intersection logic.

However, the system, which depends on the exact hash match, also makes the system vulnerable to attacks that involve rotation and cropping.

Table 2. Comparative Detection Performance

Method	Detection Rate (Recall) on Jumps	Detection Rate (Recall) on Loops	False Alarm Rate (Static)
HTM Only	High (Detected)	Low (<20% Alert Rate)	Low (<10%)
Hash Only	Low (Ignored)	High (100%)	High (False Positives w/o motion)
Fusion	High	High (100%)	Minimal (Filtered by Intersection)

The Skip to Live protocol reduced the computational lag caused by HTM's computations. Although the system ran at 8-10 frames per second on an ordinary CPU, synchronization logic helped maintain alignment with real time with an error of less than 2.0 sec. Indeed, the correlation window of 30 sec helped correlate alerts for Asynch Hash (T=0) with HTM alerts (T+), thus validating the system's real-time feasibility. Nevertheless, presently, only single stream isolation is facilitated. For each RTSP stream, there is a separate microservice instance for processing. For multi-camera scale-up, there is a need for microservice management using a container. In addition, since a fixed input image size (640x480 pixels) is considered in this system and this helps in extracting equal features from all streams, this will negatively affect equality in large-resolution streams in fine-grained manipulation detection.

V. CONCLUSION AND FUTURE WORK

This research study has proposed a new and hardware-independent method for replay attack detection within CCTV networks using a combination approach between Hierarchical Temporal Memory (HTM) and Perceptual Hashes (pHash). The project addressed the key challenge of how to coordinate different asynchronous inference engines using a Pulse Heartbeat communication mechanism coupled with a Skip to Live ingestion plan to provide real-time correlation between probabilistic motion anomalies and deterministic violations. Our results indicate this two-layer strategy achieves a robust verification system, with the HTM engine identifying "temporal physics" irregularities not seen by basic frame comparison, and the pHash auditor confirming these inconsistencies to remove false positives from environmental fluctuations. Our system is thus efficient, interpretable, and able to protect existing "brownfield" infrastructure without needing specialized cameras or GPU processing.

The future versions of the system would seek to incorporate improvements to the adaptability, resilience, and scalability of the system. Firstly, adaptive threshold would be integrated into the system, which would dynamically alter the thresholds of detection depending on the level of entropy in the camera scene, from high-action areas such as intersections to low-action areas such as corridors. Secondly, the system would incorporate a fuzzy hash comparison using a Hamming distance threshold to perceptually hash the comparisons, allowing it to detect malicious replay attacks with modifications such as crops, rotations, or noise injection to evade exact matches. Thirdly, the backend component of the system would be expanded to incorporate multiple cameras to perform spatiotemporal correlations to detect anomalies in a network of cameras, which would be necessary to secure a larger-scale surveillance system.

REFERENCES

- [1] D. Tran et al., "Learning Spatiotemporal Features with 3D Convolutional Networks," in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), 2015, pp. 4489–4497.
- [2] A. Hampapur et al., "Smart Video Surveillance: Exploring the Concept of Multiscale Spatiotemporal Tracking," IEEE Signal Process. Mag., vol. 22, no. 2, pp. 38–51, Mar. 2005.
- [3] J. C. Nascimento and J. S. Marques, "Performance Evaluation of Object Detection Algorithms for Video Surveillance," IEEE Trans. Multimedia, vol. 8, no. 4, pp. 761–774, Aug. 2006.
- [4] Zhao, Dong, Yang Shi, Steven X. Ding, Yueyang Li, and Fangzhou Fu. "Replay attack detection based on parity space method for cyber-physical systems." *IEEE Transactions on Automatic Control* 70, no. 4 (2024): 2390-2405.
- [5] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," Commun. ACM, vol. 60, no. 6, pp. 84–90, Jun. 2017.
- [6] Y. Liu, P. Ning, and M. K. Reiter, "False Data Injection Attacks Against State Estimation in Electric Power Grids," ACM Trans. Inf. Syst. Secur., vol. 14, no. 1, pp. 13:1–13:33, May 2011.
- [7] A. D. Wood et al., "Context Aware Wireless Sensor Networks for Assisted Living and Residential Monitoring," IEEE Netw., vol. 22, no. 4, pp. 26–33, Jul.–Aug. 2008.
- [8] M. A. Ferrag et al., "Security for 4G and 5G Cellular Networks: A Survey of Existing Authentication and Privacy Preserving Schemes," J. Netw. Comput. Appl., vol. 101, pp. 55–82, Jan. 2018.
- [9] Yang, Yuchen, Longfei Wu, Guisheng Yin, Lijie Li, and Hongbin Zhao. "A survey on security and privacy issues in Internet-of-Things." *IEEE Internet of things Journal* 4, no. 5 (2017): 1250-1258.
- [10] W. Sultani, C. Chen, and M. Shah, "Real-World Anomaly Detection in Surveillance Videos," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2018, pp. 6479–6488.
- [11] M. Sabokrou et al., "Deep-Anomaly: Fully Convolutional Neural Network for Fast Anomaly Detection in Crowded Scenes," Comput. Vis. Image Underst., vol. 172, pp. 88–97, 2018.
- [12] Y. Cong, J. Yuan, and J. Liu, "Sparse Reconstruction Cost for Abnormal Event Detection," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2011, pp. 3449–3456.
- [13] "UCF Crime Dataset," Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/odins0n/ucf-crime-dataset>